Jetson 카메라 응용 실습 결과보고서

Nov 2, 2026 · @김채림

요약

과제 1·2의 확인 기준은 개방 어휘 1개를 빼고 모두 충족했다. 과제 3은 붐비는 공개 데이터에서 입력 크기·임계값·보정식으로 MAE를 16.8 → 3.1명까지 줄였지만 기준(1.0명)에는 못 미쳤다. 과제 5는 가상 시험(합성 영상)으로 기준을 만족했고, 과제 6은 12시간 무중단(재시작 0회)·카메라 분리·재부팅(72초 복귀) 시험을 모두 통과했다. 심화(B)는 1080p 하드웨어 디코딩 실측(카메라 1대 CPU 65~94% → 12%), 출입+직접 혼합 보정(드리프트 +5 → 0), 기준선 6/12cm 비교(ΔZ ∝ 1/B 확인)를 추가했다.

과제	확인 기준	결과	충족
1 카메라 연동	(a)/(b) 2분 뒤 지연 비교·원인	RTSP (a) 약 96초 vs (b) 일정, USB (a) 0.83초 vs (b) 0.03초	✅
	재연결 30초 이내	4.6초, 프로세스 재시작 없음	✅
	glass-to-glass (USB·RTSP)	RTSP 408ms, USB는 원격이라 미측정	⚠️
2 물체 인식	FP16 추론 ≤ 10ms	5.09ms, 60.6fps	✅
	선 통과 10회 오차 ≤ 1	여유 폭 추가 후 0 (전 3)	✅
	벤치마크 4행, INT8 mAP 하락	4행 완성, INT8 −0.0245	✅
	개방 어휘 5종 중 4종 이상	YOLOE 최고 3/5 (임베딩 재라벨 4/5)	❌
3 재실 인원	MAE ≤ 1.0명, ±1명 ≥ 80%	Mall Dataset(평균 31명) MAE 3.09명 / 가상 방(6명) 직접 계수 MAE 0.14명·±1명 99%	⚠️
	편향 원인 설명	가림·작은 크기로 덜 셈(편향 = −MAE)	✅
	출입 vs 직접 동시 운영	가상 시험: 출입 계수 누적 오차 +1(IN·OUT 누락), 직접 계수 정답	✅
4 행동 패턴	동작 10회 중 9회, 행동 12회 중 10회, 혼동 사례	smooth로 만세·한팔·손흔들기 3/3, 깜빡임 제거 / 행동 규칙 7/12 / GRU 처음 본 사람 13/15(87%) / 혼동 사례와 원인	⚠️
5 거리 측정	가상 영상 세 지점 오차 ≤ 5%	1.5 / 2.5 / 4.0m 오차 0.1 / 0.2 / 0.0%	✅

과제	확인 기준	결과	충족
	재투영 ≤ 0.5px	0.15 / 0.19px	✅
6 응용 서비스	12시간 무중단	12시간 재시작 0회, 카메라 재연결 0회, 평균 9.1W	✅
	카메라 1대 분리 시 나머지 정상, 자동 재연결	나머지 정상, 송출 복귀 2초 안 재연결	✅
	재부팅 후 3분 안 복귀	72초 만에 서비스 응답, 카메라 정상	✅
	운영 모드 영상 저장 0건	`/health` saved_images = 0	✅
	결과 형식 문서	별도 문서(1~2쪽)	✅

가장 크게 달라진 설계 변수: 재실 인원에서 입력 크기(640 → 960)와 보정식, 선 통과에서 여유 폭(오차 3 → 0).

실습 환경

과제 1·2·6은 팀 Jetson(team2)에서 진행했다. 젯슨은 교육장에 두고 집에서 Tailscale로 원격 접속했으며, IP 카메라 대신 맥 카메라를 RTSP로 송출한 가짜 카메라를 썼다.

항목	내용
보드	Jetson Orin Nano Super (Ubuntu 24.04, TensorRT 10.16, Ultralytics 8.4)
USB 웹캠	Logitech C920 (젯슨에 직접 연결, /dev/video0)
가짜 IP 카메라	맥북 FaceTime HD 1280x720 30fps → ffmpeg(H.264, GOP 30) → mediamtx(RTSP) → ssh 역방향 터널 → 젯슨 `rtsp://localhost:8554/mac`
결과 보기	젯슨의 웹 화면(MJPEG)을 PC 브라우저로 접속 (헤드리스 운용)
측정 도구	직접 작성한 `cam1.py` (probe / lag / view), `bench2.py`, `occupancy_service.py`

젯슨에서 맥으로는 연결을 시작할 수 없는 공유 노드라서, 맥에서 젯슨 쪽으로 ssh 역방향 터널을 열어 RTSP를 전달했다.

과제 1. 카메라 연동과 기준선

최신 1장 리더(b)를 쓰면 두 입력 모두 지연이 쌓이지 않았고, 그대로 읽기(a)는 RTSP에서 2분 뒤 약 96초까지 밀렸다. 재연결은 4.6초 만에 자동 복구되어 기준(30초)을 만족했다.

입력	해상도	FPS	열기 / 첫 프레임 (ms)	프로세스 CPU	(a) 2분 뒤 지연	(b) 2분 뒤 지연	glass-to-glass	재연결
RTSP (맥 카메라)	1280x720	32.3	1442 / 1468	18.8%	약 96,000ms (계속 증가)	일정 (쌓이지 않음)	408ms (386 / 414 / 425)	4.6초
USB (C920)	1280x720	15.1	257 / 647	11.4%	833ms (일정)	31ms (일정)	미측정	미측정

- 조건: 처리 지연 0.2초, 각 120초 측정. RTSP는 2026-11-02 17:34–17:38, USB는 22:57–23:01(원격, 교육장 조명 꺼짐)에 쟀다.
- (a)의 차이: 처리(0.2초)가 카메라보다 느리면 못 읽은 프레임이 버퍼에 쌓인다. RTSP는 버퍼가 커서 끝없이 쌓였고, USB는 드라이버 버퍼가 약 4장이라 4 × 0.2초 ≈ 0.8초에서 멈췄다.
- (b)는 별도 스레드가 계속 읽고 최신 1장만 남겨 지연이 쌓이지 않는다. 대신 읽기 스레드 때문에 CPU가 늘었다(USB 7.0% → 20.1%).
- USB는 압축·네트워크 단계가 없어 첫 프레임이 RTSP보다 약 820ms 빨랐다. FPS 15.1은 어두운 환경에서 C920이 노출을 늘리려고 스스로 낮춘 값이다.
- 재연결 시험: 맥 송출을 강제로 꺼서(22:48:11) 끊겼고, 프로세스 재시작 없이 22:48:16에 복구했다.
- glass-to-glass: 젯슨 웹 화면의 밀리초 시계를 거울로 맥 카메라에 비추고, 화면 캐프처에서 '위 시계 − 영상 속 시계'를 3회 계산했다.
- USB glass-to-glass와 재연결은 젯슨이 교육장에 있어 원격으로 측정하지 못했다.

확인 문제 1

1-1. `appsink drop=true max-buffers=1 sync=false`

- `max-buffers=1` : appsink 대기열에 프레임을 1장만 보관한다.
- `drop=true` : 대기열이 차면 기다리지 않고 옛 프레임을 버리고 새 프레임을 넣는다.
- `sync=false` : 타임스탬프에 맞춰 기다리지 않고 도착 즉시 넘긴다.
- 세 옵션을 합치면 리더 (b)처럼 항상 최신 1장만 받는다.

1-2. RTSP를 UDP에서 TCP로 바꾸면

- 좋아지는 점: 패킷 손실이 없어 화면 깨짐이 없고, 방화벽·NAT·ssh 터널을 통과한다. 이번 구성(ssh -R)도 TCP라서 가능했다.
- 나빠지는 점: 손실 시 재전송을 기다리는 동안 뒤 프레임도 멈춰 지연이 튀고, 네트워크가 나쁘면 버퍼가 쌓인다.

1-3. GOP를 FPS와 같게(1초) 두는 이유

- 디코더는 I-프레임부터만 화면을 만든다. 처음 접속하거나 재연결하면 다음 I-프레임까지 기다려야 하는데, GOP가 1초면 이 대기가 최대 1초다.
- GOP를 길게 하면 대역폭은 줄지만 첫 화면과 복구가 그만큼 늦다. 이번 송출도 `-g 30`(30fps에서 1초)이었다.

1-4. 보드 CPU 부하가 가장 작은 카메라

- CSI 카메라. 센서 영상이 MIPI로 젯슨의 VI·ISP 하드웨어에 바로 들어와 압축 풀기·USB·네트워크 처리가 없다.
- USB는 USB 처리와 MJPEG 풀기, IP 카메라는 네트워크와 H.264 풀기가 CPU에 든다. 측정에서도 프로세스 CPU가 USB 11.4% < RTSP 18.8%였다.

1-5. 리더 (b)에서도 e2e 지연이 300ms일 때 코드가 줄일 수 없는 부분

- 카메라 안: 센서 노출 시간(30fps면 약 33ms)과 카메라 내부 처리·압축.
- 전송: 네트워크 이동 시간.
- 표시: 모니터 주사율(60Hz면 최대 약 17ms)과 브라우저 그리기.
- 코드가 줄일 수 있는 것은 대기열과 추론 시간뿐이고, 대기열은 (b)로 이미 없앰다.

과제 2. 물체 인식 — TensorRT 벤치마크

TensorRT FP16 엔진의 추론 시간은 5.09ms로 기준(10ms 이하)을 만족했고, 전체 처리 60.6fps로 카메라(30fps)를 충분히 따라간다. INT8은 FP16보다 8%만 빠르고 mAP50-95가 0.0245 떨어져 기본값으로 쓰지 않는다.

모델	추론 평균 (ms)	추론 p95 (ms)	전체 평균 (ms)	FPS	mAP50-95	mAP50	전력 (W)	파일 (MB)
PyTorch yolo11n (FP32)	25.14	26.09	32.69	30.6	0.5076	0.6749	12.41	5.6
TensorRT yolo11n FP16	5.09	6.88	16.50	60.6	0.5063	0.6757	11.60	8.5
TensorRT yolo11n INT8	4.66	6.66	16.70	59.9	0.4818	0.6540	10.80	5.8
TensorRT yolo11s FP16 (큰 모델)	8.87	12.67	21.06	47.5	0.5765	0.7460	13.21	22.0

측정 조건 — 입력 640, batch 1, bus.jpg 300회 반복(예열 30회 제외), mAP는 coco128(128장), 전력은 tegrastats VDD_IN 평균. INT8 보정은 coco128로 85초. 과제 6 서비스가 같은 GPU에서 동시 가동 중, 보드 RAM 사용 약 6.5~6.7GB. 2026-11-02, `bench2.py`*, 총 1,718초.*

- PyTorch → FP16: 추론은 약 5배(25.14 → 5.09ms) 빨라졌지만 전체는 2배(32.69 → 16.50ms)다. 남은 약 11ms는 CPU에서 하는 전처리(크기 조절)와 후처리(NMS)다. mAP는 거의 같다(0.5076 → 0.5063).
- INT8: FP16 대비 추론 0.43ms(8%) 빠르고 전력 0.8W 적지만, mAP50-95가 0.5063 → 0.4818(약 4.8%) 떨어졌다. 작은 모델이라 속도 이득이 작아 정확도 손해를 감수할 이유가 없다.
- 큰 모델(yolo11s) FP16: mAP50-95가 0.0702 오르고 추론 8.87ms로 여전히 10ms 이하다. 먼 사람을 봐야 하는 재실 인원처럼 정확도가 중요하면 이쪽이 낫다.
- 한계: 프로세스 메모리는 네 모델을 한 프로세스에서 차례로 재서 최대값(3,595MB)만 남아 모델별로 구분되지 않는다. 다음에는 모델마다 따로 실행해 재야 한다.

조명 × 거리별 사람 검출 신뢰도 (합성 시험)

밝기가 30% 이상이면 12m까지 88% 이상 검출했고, 밝기 10%부터 5m 이상에서 무너지기 시작해(68~73%) 밝기 5%에서는 8m 이상에서 거의 못 찾았다(0~2%). 어두움이 거리보다 훨씬 큰 변수다.

거리 (사람 키 px)	밝음 100%	어둑 30%	어두움 10%	아주 어두움 5%
2m (818px)	100% · 0.84	100% · 0.85	100% · 0.83	50% · 0.44
5m (327px)	100% · 0.82	100% · 0.78	68% · 0.61	35% · 0.46
8m (204px)	100% · 0.74	100% · 0.71	73% · 0.55	2% · 0.21
12m (136px)	98% · 0.63	88% · 0.57	55% · 0.42	0% · 0.11

칸 = 검출률 · 평균 신뢰도. 합성 시험(실측 아님) — 서 있는 사람 3명 사진(bus.jpg)을 1280x720 화면에 거리별 크기(키 1.7m, 맥 카메라 fy 962 기준)로 배치하고, 밝기를 낮추며 어두울수록 큰 센서 노이즈를 넣었다. 조건마다 노이즈·위치를 바꾼 20장, yolo11n FP16, 검출 = IoU≥0.5 & conf≥0.4. 2m는 사람이 화면보다 커서 머리·다리가 잘린다. 2026-11-03, `light2.py`.

- 신뢰도는 거리가 멀수록(0.84 → 0.63), 어두울수록 떨어진다. 조명이 충분하면 사람 키가 136px(12m)이어도 거의 다 잡힌다.
- 어두운 화면은 노이즈가 윤곽을 덮어 먼 사람부터 사라진다. 10%에서 5m(68%)가 8m(73%)보다 낮은 것은 20장 노이즈 차이 수준이다.
- 현장 적용: 야간·소등 후 교실은 IR 카메라나 보조조명이 필요하고, 재실 인원처럼 먼 사람을 봐야 하면 conf를 낮추거나 입력을 키워야 한다(과제 3 결과와 같은 방향).
- 한계: 사진 1장을 키운·줄인 합성이라 실제 원근(흐림, 압축)과 조명 방향은 반영하지 못한다. 교육장에서 2/5/8m 실측으로 확인해야 한다.

추적 + 가상 선 통과 (IN/OUT)

선 양쪽에 여유 폭(화면 너비 6%)을 두자 실제 IN 5 / OUT 5를 정확히 셌다(오차 0, 기준 ≤1). 여유 폭이 없을 때는 같은 시험에서 IN 7 / OUT 6으로 3번 더 셌다.

판정 방식	실제 IN / OUT	센 IN / OUT	오차
박스 하단 중앙 + 외적 부호 (여유 폭 없음)	5 / 5	7 / 6	3
+ 선 양쪽 여유 폭 6% (히스테리시스)	5 / 5	5 / 5	0

측정 조건 — 맥 카메라 RTSP 1280x720, TensorRT yolo11n FP16 + ByteTrack, 화면 가운데 세로선 (위→아래로 그음, 왼→오 = IN), 약 42fps. 통과할 때마다 MQTT `aiot/space/lab1/line` (QoS 1)으로 발행하고 `runs/line2_log.jsonl` 에 기록. 2026-11-03, `line2.py`.

- 원인: 카메라에 가까워 박스가 화면을 꽉 채우고 판정점(박스 하단 중앙)이 화면 아래 끝에 붙어 있었다. 선 근처에서 박스가 조금만 흔들려도 판정점이 선을 넘었다 돌아와 같은 사람을 여러 번 셌다(확인 문제 2-4의 경우).
- 수정: 여유 폭 안에서는 판정을 보류하고, 반대쪽 여유 폭을 완전히 넘었을 때만 1번 센다. 코드는 `--margin` 옵션 하나로 바꿨다.

COCO에 없는 물품 5종 — 세 방식 비교

임베딩 재라벨(②)이 4/5로 가장 정확했고, 개방 어휘 YOLOE(③)는 프롬프트·임계값을 바꿔도 최고 3/5로 기준(4/5 이상)에 못 미쳤다. YOLO 라벨 그대로(①)는 0/5로, 모르는 물품에 비슷한 COCO 라벨을 붙였다.

물품 (프롬프트)	① YOLO 그대로	② 임베딩 재라벨 (유사도)	③ YOLOE 1차 (임계값 0.15)	③ YOLOE 2차 (0.05 + 프롬프트 수정)
빗 (hair brush)	toothbrush / tv	O (0.659)	O 10/10	—
약통 (pill bottle)	bottle	O (0.676)	O 14/14	—
보조배터리 (power bank → portable battery pack)	cell phone / remote	O (0.641 / 0.631)	없음 (1/9)	파우치 (오인식)
파우치 (cosmetic pouch → small zipper bag)	frisbee	O (0.560 / 0.561)	없음	O 10/10
충전기 (phone charger → white usb wall charger)	없음 / couch	X (없음 / 파우치)	없음	보조배터리 (오인식)
정답	0/5	4/5	2/5	최고 조합 3/5

측정 조건 — 맥 카메라 RTSP 1280x720, 물품을 화면 가운데 가까이 들고 8초 동안 매 프레임 판정, 가장 많이 나온 답을 기록(9~121프레임). ① TensorRT yolo11n FP16(conf 0.25, 사람 제외, 크고 가운데인 박스), ② ①의 박스를 EmbeddingGemma 2로 임베딩해 5개 텍스트와 비교(유사도 0.20 미만이면 YOLO 라벨 유지), ③ yoloe-11s-seg에 5개 클래스를 텍스트로 지정. 2026-11-03, `vocab2.py`*, 결과* `runs/vocab2.csv`*,* `runs/vocab2_r2.csv`*, 스냅샷* `runs/vocab2_*.jpg`*.*

- ①의 오인식은 모양이 비슷한 COCO 클래스로 간다: 약통→bottle, 보조배터리→cell phone·remote, 빗→toothbrush, 둥글고 납작한 파우치→frisbee.
- ②는 YOLO가 박스만 잡으면 잘 맞힌다. 하지만 YOLO가 못 잡은 충전기는 재라벨할 박스가 없어 실패했고, 엉뚱한 박스(couch)를 잡으면 그 박스를 재라벨해 틀렸다. 유사도는 0.56~0.68로 임계값 0.20보다 훨씬 높아, 후보에 없는 물체도 걸러내지 못할 것이다.
- ③는 모양이 뚜렷한 물품(빗, 약통)은 확실히 맞히지만, 네모난 작은 상자류(보조배터리, 충전기)는 헷갈렸다. 임계값을 0.05로 낮추자 '없음'이 줄었지만, 대신 다른 후보 이름을 자신 있게 붙였다(보조배터리→파우치, 충전기→보조배터리). 프롬프트 문구를 바꾸자 파우치는 없음 → 정답으로 바뀌었다.
- 안경을 처음 후보로 넣었을 때는 측정자가 쓴 안경을 대신 잡아 빗을 '안경'으로 96/97 오인식했다. 후보 목록에 현장에 항상 보이는 물체가 있으면 오인식이 생긴다.
- 다음 시도: 물품마다 현장 사진을 모아 YOLOE의 시각 프롬프트(예시 이미지)를 쓰거나, ②와 ③를 합쳐 ③가 잡은 박스를 ②로 재확인한다.

남은 2번 항목: 조명×거리 신뢰도 표(교육장).

확인 문제 2

2-1. `.pt` → `.engine` 변환이 빨라지는 이유 세 가지, 다른 보드에 복사하면 안 되는 이유

- 층 합치기: Conv·BN·활성화 같은 연속 연산을 하나의 커널로 묶어 GPU 호출과 메모리 왕복을 줄인다.
- 정밀도 낮추기: FP32를 FP16/INT8로 바꿔 계산량·메모리를 줄이고 Tensor Core를 쓴다.
- 커널 자동 선택: 빌드 때 이 GPU에서 계산 방식을 실제로 재 보고 가장 빠른 것을 고르며, 입력 크기를 고정해 메모리 배치도 미리 정한다.
- 복사 금지: 엔진은 만든 GPU 구조와 TensorRT·CUDA 버전에 맞춰 고른 커널이 들어 있다. 다른 보드나 버전에서는 열리지 않거나 느려서, 그 보드에서 다시 빌드해야 한다.

2-2. 평균보다 p95 지연을 함께 보는 이유

- 상시 서비스는 매 프레임을 제시간에 처리해야 한다. 평균은 가끔 생기는 느린 프레임을 가린다.
- 느려지는 원인: 다른 프로세스와 GPU 공유(이번에도 과제 6 서비스가 동시에 가동), 발열로 인한 클럭 저하, 메모리 정리.
- 10fps에서 5%는 1분에 30장이다. 그 순간에 프레임이 밀리거나 알람이 늦어지므로 꼬리 지연을 보고 여유를 정해야 한다.

2-3. 박스 하단 중앙을 쓰는 이유, 코드 없이 IN·OUT을 바꾸는 법

- 하단 중앙은 사람의 발, 즉 바닥에서 서 있는 위치다. 바닥에 그은 선과 같은 평면에서 비교된다.
- 박스 중심은 키·자세(앉기, 팔 뻗기)·가림·원근에 따라 위치가 흔들려, 실제로 선을 넘지 않아도 넘은 것처럼 보일 수 있다.
- 방향 바꾸기: 선의 두 끝점 순서를 바꿔 아래→위로 긋는다. 외적 부호가 반대가 되어 왼→오 이동이 OUT이 된다. 이번 `line2.py`에서는 `--line 0.5,0.9,0.5,0.1`로 바꾼다.

2-4. 선 위에서 멈췄다가 되돌아간 경우

- 발이 선을 넘었다 돌아오면 IN 1, OUT 1이 된다. 실제로 들어왔다 나간 것과 같고 순인원은 0이라 버그는 아니다.
- 다만 선 위에서 박스가 흔들리면 IN·OUT이 여러 번 쌓일 수 있다. 선 양쪽에 여유 폭(히스테리시스)을 두거나 여러 프레임 연속으로 넘어야 인정하면 줄어든다.

2-5. `relabel` 모드에서 유사도 임계값 0.20을 없애면

- 유사도는 항상 후보 중 가장 높은 것을 고르므로, 후보에 없는 물체도 가장 비슷한 라벨을 강제로 받는다.
- 그러면 모르는 물체에 틀린 라벨을 자신 있게 붙이는 문제가 오히려 늘어난다. 임계값은 근거가 약하면 원래 YOLO 라벨을 유지하는 안전장치다.

과제 3. 재실 인원 카운터 — 정확도 평가

YOLO는 붐비는 실내에서 사람을 항상 덜 셌고(서비스 기본값 MAE 16.2명, 평균 31명 중), 입력 크기·임계값을 바꾸고 직선 보정식을 더하자 시험 구간 MAE가 3.1명까지 줄었다.

조건	MAE (명)	편향 (명)	±2명 이내	속도 (ms/장)	시험 MAE 보정 전 → 후	보정 후 ±2명 이내
yolo11n FP16 · 640 · conf 0.4 (서비스 기본값)	16.20	−16.20	0.0%	21.9	16.81 → 3.90	41.4%
yolo11n FP16 · 640 · conf 0.25	12.78	−12.78	0.0%	20.2	13.20 → 3.20	47.6%
yolo11s FP16 · 640 · conf 0.4	17.22	−17.22	0.0%	24.3	17.80 → 4.62	33.5%
yolo11s FP16 · 640 · conf 0.25	13.75	−13.75	0.1%	25.5	14.11 → 3.93	38.3%
yolo11s PyTorch · 960 · conf 0.25	10.35	−10.35	0.9%	82.2	10.70 → 3.09	48.7%

측정 조건 — Mall Dataset(CUHK) 2,000장, 640x480 실내 고정 카메라, 프레임마다 사람 수 정답(13~53명, 평균 31.2명). 사람(class 0)만 센다. 편향 = 센 값 − 정답. 보정식(정답 ≈ a × 센 값 + b)은 앞 1,000장으로 만들고 뒤 1,000장에서만 채점했다(예: 서비스 기본값 a = 1.54, b = 7.5). 중앙값 5장 평활은 MAE를 거의 바꾸지 않아 표에서 냈다. 2026-11-03, `occ_eval.py`, `occ_calib.py`.

- 왜 덜 세나: 위에서 내려다보는 각도에 먼 사람은 몇십 픽셀로 작고 서로 겹친다. 정답은 머리가 보이는 사람 전부인데 YOLO는 몸이 충분히 보여야 박스를 그린다. 편향이 MAE와 같아 흔들림이 아니라 꾸준한 누락이다.

- 가장 큰 설계 변수는 입력 크기다. 640 → 960으로 키우자 MAE 13.75 → 10.35로 줄었고, 대신 3배 이상 느려졌다(1초에 1번 세는 서비스에는 충분). conf를 0.4 → 0.25로 낮추는 것도 효과가 있었다(16.20 → 12.78). 같은 조건에서 모델만 키우는 것(n → s)은 효과가 없었다.
- 보정식은 꾸준한 누락을 거의 없앰다(편향 −1명 이내). 단, 카메라 각도·방 구조가 바뀌면 a, b가 달라지므로 방마다 정답을 조금 세어 다시 구해야 한다.
- 과제 6 서비스에 반영할 설정: conf 0.25, 입력 960, 방별 보정식. 교육장 실측(사람 수를 직접 센 10회)으로 다시 확인해야 한다.

출입 계수 vs 직접 계수 — 가상 시험(합성 영상)

두 방식을 같은 영상에서 동시에 돌리자 직접 계수는 MAE 0.14명으로 정답을 거의 따랐고, 출입 계수는 놓친 IN·OUT이 쌓여 끝났을 때 1명 틀렸다(정답 4, 출입 5).

방식	MAE (명)	편향 (명)	±1명 이내	끝났을 때 (정답 4)
직접 계수 (방 안 발 위치, 중앙값 5프레임)	0.14	−0.13	99%	4
출입 계수 (문 선 IN−OUT 누적)	0.63	−0.29	99%	5

가상 시험(합성 영상) — 카메라 1대(1280x720, 높이 2.5m, 25° 아래), 사람 6명이 차례로 들어오고(두 명은 붙어서, 한 명은 앞사람 뒤에 서서 가려짐) 2명이 나감. yolo11n + ByteTrack, conf 0.3, 문 선 여유 폭 0.25m, 프레임 간격 0.2초, 56초. 2026-11-03, `vcount.py`*.*

- 출입 계수는 IN 5/6, OUT 0/2를 셌다. 두 명이 붙어 들어올 때 한 명이 가려져 IN을 하나 놓쳤고(−1), 나간 두 명의 OUT을 모두 놓쳐(+2) 끝에 +1이 남았다. OUT을 놓친 원인은 확인하지 못했다(문 근처 추적 ID 끊김 추정).
- 출입 계수의 오류는 사라지지 않고 누적된다. 직접 계수는 가려진 순간(8초, 정답 2 → 1)에만 틀리고 다시 돌아온다.
- 선택: 방 안이 한 화면에 보이면 직접 계수를 기본으로 하고, 출입 계수는 방이 너무 커서 다 안 보일 때 보조로 쓰되 직접 계수로 주기적으로 누적 값을 바로잡는다. 붐비는 공간(Mall Dataset)에서는 직접 계수도 크게 덜 셈으므로 보정식이 필요하다.

과제 4. 행동 패턴 인식 (9단계 결과 재사용)

YOLO 포즈 키포인트로 동작(손흔들기·박수·팔짱·가만히)과 행동(관심 등)을 규칙과 학습(GRU) 두 방식으로 판정했다. 학습 모델이 '가만히(idle)'를 내면 규칙의 손흔들기를 덮어쓰던 문제를 고쳤다.

항목	내용
프로그램	`pose_app.py` (실시간 인식 + 웹 대시보드), `train_pose.py` (GRU 학습)

항목	내용
동작 모델	`models/action.pt` — 라벨 arms_crossed, clap, idle, wave, 창 30프레임
행동 모델	`models/behavior.pt`
혼동 사례 1	학습 모델이 idle로 판단하면 규칙 기반 손흔들기를 덮어써 손흔들기가 잡히지 않았다 → idle일 때는 규칙 결과를 유지하도록 수정
혼동 사례 2	영어 라벨(interested 등)이 한글 순위표와 맞지 않아 관심 비율이 0%로 나왔다 → 라벨을 한글로 변환
혼동 사례 3	손을 연속으로 흔들면 1회로 기록(동작이 바뀔 때만 기록) → 따로 세려면 약 2초 손을 내려야 한다

raw vs 안정화(smooth) — 동작마다 3회씩, 센 횟수:

동작 (실제 3회)	raw	smooth	비고
만세	3	3	raw는 올리고 내릴 때 한팔들기로 6번 깜빡임, smooth는 깜빡임 없음
한팔들기	3	3	둘 다 정확
손흔들기	0	3	raw는 모두 한팔들기로 인식, smooth는 1~1.5초 뒤 손흔들기로 바뀜(판정 지연)
악수	5~6	5	맞잡고 가만히 있으면 손목이 가려져 미검출, 두 사람이 겹치기만 해도 악수로 오인식
측면 만세	6	5	옆모습에서 반대쪽 어깨가 가려져 과다 카운트

행동 패턴(규칙): 지나감 3/3, 힘끗 봄 1/3, 관심 있음 3/3, 머무름 0/3 → 12회 중 7회(기준 10회 미달). 규칙이 시선이 아니라 머리 방향을 봐서, 걸으며 고개를 돌리면 정면 시간이 짧게 잡혔고(힘끗 봄→지나감), 옆을 보며 서 있으면 한쪽 어깨가 가려져 판정이 갱신되지 않았다(머무름→힘끗 봄).

학습 모델(GRU): 동작 모델 최고 검증 정확도 0.976(팔짱 30개 중 3개를 손흔들기로 오판), 행동 모델 1.000(지나감 검증 조각 2개라 참고 수준). 녹화에 참여하지 않은 사람에게 15번 중 13번(87%) 맞혔고, 약점은 빠른 박수(1/3, 팔짱으로 오판)였다. 규칙으로 불가능한 박수·팔짱을 학습이 인식했다.

측정 조건 — 9단계(embedding_gemma2 실습) 결과 재사용. USB 웹캠, yolo11n-pose, smooth = 0.6초 다수결 + 1.5초 손목 움직임, GRU window 30. 2026-10-26.

과제 5. 거리 측정 (카메라 2대) — 가상 시험(합성 영상)

가상 카메라 2대와 합성 영상으로 시험했다. 어깨 삼각측량(stereo)은 모든 시나리오에서 평균 오차 0.09~0.10m였고, 발목 방식(ankle1)은 평균 0.55~0.66m 틀렸으며 다리가 가려지자 위치를 전혀 내지 못했다(0/30).

시나리오	ankle1 (발목 → 바닥)	stereo (어깨 삼각측량)	fused (stereo + 한쪽 추정)
S1 트인 곳 걷기 (8m → 1.5m)	38/40 · 0.66m (최대 1.20)	40/40 · 0.10m (최대 0.15)	40/40 · 0.10m (최대 0.15)
S2 책상 뒤 걷기 (다리 가림)	0/30 · —	30/30 · 0.09m (최대 0.12)	30/30 · 0.09m (최대 0.12)
S3 옆으로 비켜 감 (한 카메라에만)	24/32 · 0.55m (최대 0.64)	23/32 · 0.09m (최대 0.11)	23/32 · 0.09m (최대 0.11)

칸 = 위치가 나온 프레임 · 평균 오차(바닥 위치 x, y). 가상 시험(합성 영상) — 가상 카메라 2대 (1280x720, 초점거리 960px, 높이 1.2m, 간격 1.0m, 10° 아래)가 본 합성 화면에 yolo11n-pose를 실제로 돌렸다. 정답 위치를 알아 오차를 바로 채점했다. 키포인트 신뢰도 0.5, fused의 한쪽 추정 높이 1.2m. 2026-11-03, `vstereo.py`*.*

- 두 카메라 어깨 삼각측량은 발이 가려져도 위치를 잡고, 오차도 ankle1의 약 1/6이다. 발목 방식은 발목 점이 조금만 어긋나도 바닥과 만나는 점이 크게 밀린다.
- S3에서는 fused가 stereo보다 더 많이 잡지 못했다(23/32로 같음). 한 카메라에만 보이는 위치에서는 사람이 화면 끝에 걸려 어깨 신뢰도가 0.5를 넘지 못했기 때문으로 보인다. fused의 보완 효과는 이 시험으로는 확인하지 못했다.

세 지점 거리 오차와 접근·충돌 알람 — 가상 시험(합성 영상)

가상 영상 세 지점의 거리 오차는 0.0~0.2%로 기준(5% 이하)을 만족했고, 고정 타겟 알람은 정답과 0.1초 안에서 울렸다.

실제 거리 (m)	측정 평균 (m)	오차
1.5	1.502	0.1%
2.5	2.505	0.2%
4.0	3.999	0.0%

고정 타겟 접근 알람 (10-6): 근접 1.5m, 해제 1.7m(히스테리시스 0.2m). 접근 2.6초(정답 2.6초, 측정 거리 1.47m), 이탈 7.7초(정답 7.6초, 1.71m). 알람 2회로 헛알람 없음.

이동 타겟 충돌 예상 시간 (10-8): 다른 사람(타겟)이 서 있는 사람에게 다가옴.

실제 속도 (m/s)	추정 속도 (m/s)	충돌 위험 알람 때 거리 (m)	근접 알람 때 거리 (m)	충돌 위험이 먼저 울린 시간 (s)
0.5	0.49	0.82	1.47	−1.2 (늦음)
1.0	1.00	1.94	1.47	0.5
1.5	1.49	2.89	1.39	1.0

가상 시험(합성 영상) — 카메라 2대(높이 1.2m, 간격 1.0m), 사람 2명, 두 카메라 사람 짝은 재투영 오차 15px 이내에서 가장 작은 쌍, 속도 = 최근 1초 위치의 직선 기울기, 충돌 위험 = 접근 속도 > 0.5m/s 이고 TTC < 2초, 프레임 간격 0.1초. 2026-11-03, `vprox.py`*.*

- 빠를수록 충돌 위험 알람이 더 멀리서, 더 일찍 울린다(1.5m/s에서 2.89m, 근접 알람보다 1.0초 먼저). 거리만 보는 근접 알람은 속도와 상관없이 1.4~1.5m에서 울린다.
- 0.5m/s는 접근 속도가 기준(0.5m/s)과 거의 같아 충돌 위험이 근접보다 1.2초 늦게, 0.82m에서야 울렸다. 천천히 다가오는 대상은 근접 알람이 막아야 한다.
- 추정 속도는 실제와 ±0.01m/s 안이다. 가상 시험은 두 카메라 시간 차(skew)가 0이라, 실제 웹캠(약 80ms)에서는 1m/s에 약 8cm 위치 오차가 더 생긴다.

과제 6. 응용 서비스 (재실 인원)

재실 인원 서비스 v1을 systemd 서비스로 등록해 2026-11-02 약 18:01부터 12시간 연속 가동 중이다(종료 예정 11-03 06:01).

항목	내용
입력	교육장 VIGI IP 카메라 (stream2), 최신 프레임만 유지, 끊기면 2초마다 재연결
추론	YOLO yolo11n, 1초마다 사람(class 0) 수 계산, 30초 창 중앙값으로 평활
발행	10초마다 MQTT `aiot/space/lab1/occupancy` (retain, QoS 1), 상태 `aiot/service/occupancy/status`
REST	`:8095/occupancy`, `:8095/health` (CPU·메모리·GPU, 저장 이미지 0장)
기록	1분마다 `logs/service_날짜.jsonl`, 이벤트 `logs/events.jsonl`, `tegrastats.log`

항목	내용
자동 복구	`Restart=always`, 비밀번호는 EnvironmentFile(chmod 600)에 보관
개인정보	영상·이미지는 저장하지 않고 인원 숫자만 보드 밖으로 내보냄

가동 직후 확인: count 0, 카메라 정상, 재연결 0회.

12시간 가동 기록

11-02 18:01부터 12시간 동안 서비스는 한 번도 재시작하지 않았고, 카메라는 718분 중 717분 정상, 재연결 0회, MQTT는 내내 연결돼 있었다.

항목	값
기간	2026-11-02 18:01 ~ 11-03 06:01 (12시간)
기록	1분마다 718행 / 기대 720행, 최대 공백 62초
서비스 재시작	0회 (가동 시간 초기화 0회, 시작 이벤트 1회)
카메라 lab1	정상 717/718행, 재연결 0회
MQTT 연결	718/718행
추론 시간	평균 42.7ms / 최대 50.3ms
보드 CPU	평균 25.5% / 최대 91.7%
보드 메모리	평균 4,163MB / 최대 6,988MB
보드 GPU	평균 68.7% / 최대 99.7%
전력 (VDD_IN)	평균 9.1W / 최대 18.3W
재실 인원	평균 0.0명 / 최대 0.0명 (야간 빈 교육장)

측정 조건 — VIGI IP 카메라 stream2, yolo11n PyTorch 입력 640, conf 0.4, 1초에 1번 세기, 30초 중앙값, 10초마다 발행. CPU·메모리·GPU·전력은 보드 전체 값이다. 같은 밤 23:00~01:35에 같은 보드에서 벤치마크·가상 시험을 함께 돌려 최대값과 평균 GPU가 그만큼 높게 나왔다. `occ_summary.py`.

- systemd의 NRestarts는 15로 나오지만, 이는 18:01 이전 포트 8090 충돌로 시작에 실패하던 재시도까지 센 값이다. 12시간 안에는 가동 시간이 한 번도 초기화되지 않았다.
- 빈 방 오판: 야간 12시간 718번 발행 모두 0명으로, 빈 방을 '있음'으로 본 경우가 없었다(0%, 기준 10% 이하). 단 조명이 꺼진 상태일 수 있어, 포스터·모니터 속 사람 같은 오검출은 낮 시간에 다시 확인해야 한다.

자원 계획 (계산치 vs 실측치)

자원	계산	계산치	실측치
GPU (이 서비스)	카메라 1대 × 1회/초 × 추론 42.7ms	약 4.3%	보드 전체 평균 68.7% (다른 실험 포함)
GPU (FP16 엔진으로 바꾸면)	1 × 1 × 5.1ms (과제 2 벤치마크)	약 0.5%	—
카메라 늘리기	10대 × 1회/초 × 42.7ms	약 43%	—
전력	—	—	평균 9.1W

- 추론 시간 42.7ms는 벤치마크의 PyTorch 25ms보다 길다. 같은 GPU를 다른 실험과 나눠 썼기 때문이다. FP16 엔진으로 바꾸면 카메라를 여러 대 늘려도 GPU 여유가 충분하다.
- 보드 메모리는 최대 6,988MB(전체 약 7.5GB)까지 올랐다. 카메라를 늘리거나 LLM을 함께 올리면 GPU보다 메모리가 먼저 바닥난다.

재부팅 복구 시험

보드를 재부팅하자 72초 만에 서비스가 자동으로 다시 떠서 REST에 응답했고, 카메라도 정상으로 인원을 세고 있었다(기준 3분 이내).

항목	값
재부팅 명령	2026-11-03 08:52:06 (`sudo reboot`)
복귀 확인	`:8095/health` 첫 응답까지 72초
복귀 직후 값	lab1 count 0, camera 정상, age_s 0.0, reconnects 0 (08:53:10 발행)

측정 — 맥에서 2초마다 `curl` 로 `:8095/health` 를 확인해 첫 응답 시각을 쟀다(재부팅 명령 직후 시작). 자동 시작은 systemd `WantedBy=multi-user.target`, 카메라 비밀번호는 EnvironmentFile에서 읽는다.

카메라 1대 분리 시험

카메라 하나를 끊어도 나머지 카메라는 계속 정상으로 셈고, 끊긴 카메라는 송출이 돌아오자 프로세스 재시작 없이 자동으로 다시 연결됐다.

시각	사건	lab1 (VIGI)	home (맥 카메라 RTSP)
01:28:25	시험 서비스 시작 (카메라 2대)	정상	정상
01:30:48	home 송출·터널 강제 종료 (카메라 분리)	정상 (계속 셈)	끊김(재연결 중)
01:33:03	home 송출 다시 시작	정상	복구 (끊긴 지 134.3초)
01:33:03~01:33:15	송출이 안정되기 전 끊김·복구 5번 반복 (각 1.9초 만에 복구)	정상	이후 정상, 재연결 6회

시험 조건 — 12시간 서비스는 그대로 두고 같은 코드로 시험용 서비스를 따로 띄움(포트 8098, 토픽 `test/space`). 분리는 맥의 RTSP 송출과 ssh 터널을 강제 종료해 대신했다. 기록 `logs/events.jsonl`. 2026-11-03.

- 134.3초는 송출을 다시 켜기까지 기다린 시간이고, 서비스는 송출이 돌아온 뒤 2초 안에 다시 받기 시작했다.
- 복구 직후 12초 동안 끊김·복구가 반복돼 재연결 횟수가 6으로 부풀려졌다. 송출 쪽(ffmpeg·터널)이 자리 잡기 전에 열려 읽기가 실패한 것으로 보인다. 개선: 일정 시간(예: 5초) 안정적으로 프레임이 들어와야 '복구'로 기록하고, 재연결 간격을 점점 늘린다.

심화(B) 항목

과제별 심화 항목 중 원격으로 할 수 있는 3개를 했다. 1080p 디코딩 비교는 젯슨에서 실측했고, 혼합 보정과 기준선 비교는 가상 시험(합성 영상)이다. 과제 4의 GRU 학습(처음 본 사람 13/15)은 과제 4 절에 있다.

과제	심화 항목	방법	결과
1	1080p 하드웨어(NVDEC) vs CPU 디코딩	실측	실시간 30fps 1대분 CPU 94%(GStreamer CPU)·65%(OpenCV) → 12%, 최대 180fps vs 98·157fps
3	출입 계수를 직접 계수로 주기 보정 (혼합)	가상 시험	드리프트 +5 → 0, MAE 1.73 → 0.81명
5	기준선 6cm / 12cm, $\Delta Z \propto 1/B$	가상 시험	오차 비 평균 약 2.2(이론 2), 12cm만 모든 거리 기준 통과

과제 1 심화 — 1080p 디코딩: 하드웨어(NVDEC) vs CPU (실측)

카메라 1대분(1080p 30fps)을 받을 때 하드웨어 디코딩은 CPU를 12%만 썼다. 같은 GStreamer의 CPU 디코딩(94%)보다 약 8배, OpenCV CPU 디코딩(65%)보다 약 5배 적다. 최대 속도도 하드웨어가 가장 빨랐다(180fps).

방식	흘리는 속도	처리 FPS	프로세스 CPU (코어 1개 = 100%)	걸린 시간 (s)
GStreamer nvv4l2decoder (하드웨어, NVDEC)	최대 속도	180	64%	5.0
GStreamer nvv4l2decoder (하드웨어, NVDEC)	실시간 30fps	30	12%	30.2
GStreamer avdec_h264 (CPU)	최대 속도	98	226%	9.2
GStreamer avdec_h264 (CPU)	실시간 30fps	30	94%	30.2
OpenCV VideoCapture (FFmpeg, CPU)	최대 속도	157	266%	5.7
OpenCV VideoCapture (FFmpeg, CPU)	실시간 30fps	30	65%	30.0

측정 조건 — 1920x1080 30fps H.264 High 30초(900프레임, GOP 30, B-프레임 없음, testsrc2 + 노이즈), 젯슨 MAXN_SUPER. 처리 FPS = 900 ÷ 걸린 시간. GStreamer는 두 방식 모두 BGRx까지 변환한 뒤 버림(OpenCV로 넘길 수 있는 형태). CPU 사용률 = 그 프로세스의 (user+sys) CPU 시간 ÷ 걸린 시간. 같은 시험을 먼저 한 번 돌렸을 때는 OpenCV 실시간 53%·최대 138fps, 하드웨어 12%·167fps로 판단은 같았다(보드를 팀과 함께 써서 실행마다 조금씩 다름). 2026-11-03, `dec_compare.py`.

- CPU 디코딩은 1080p 1대에 코어 0.65~0.94개를 쓴다. 6코어 보드에서 카메라 6~9대면 디코딩만으로 CPU가 찬다. 하드웨어 디코딩(코어 0.12개)이면 훨씬 많이 받을 수 있다.
- GStreamer CPU 방식이 OpenCV보다 더 무거운 것은 videoconvert(YUV → BGRx 색 변환)까지 CPU가 하기 때문으로 보인다. 하드웨어 방식은 색 변환도 nvvidconv(VIC)가 한다.
- 한계: 지금 앱(cam1.py, 재실 서비스)이 쓰는 pip 판 OpenCV에는 GStreamer가 빠져 있어 하드웨어 디코딩을 바로 못 쓴다. GStreamer appsink를 직접 연결하거나 GStreamer를 넣어 OpenCV를 다시 빌드해야 한다.
- Windows 웹캠 송출 스크립트(`win/start_win.ps1`, 맥용 `mac/start_mac.sh`를 옮긴 것)는 만들어 두었고, 노트북에서 시험은 아직 못 했다.

과제 3 심화 — 혼합 보정: 출입 계수 + 직접 계수 (가상 시험)

출입 계수는 놓친 IN·OUT이 쌓여 끝에 5명이 남았지만, 30초마다 직접 계수로 바로잡는 혼합 방식은 드리프트를 0으로 없애고 MAE도 셋 중 가장 낮았다.

방식	MAE (명)	편향 (명)	±1명 이내	최대 오차	끝났을 때 오차 (드리프트)
① 직접 계수 (5프레임 중앙값)	1.11	−1.07	66%	5	0
② 출입 계수 (IN−OUT 누적)	1.73	+1.69	56%	5	+5
③ 혼합 (30초마다 보정)	0.81	−0.72	79%	4	0

가상 시험(합성 영상) — '압축한 하루' 10분에 30명이 무작위 시각에 들어와 40~300초 머물다 나감 (20%는 앞사람과 붙어서 들어옴). 카메라 1대 1280x720 높이 2.5m 25° 아래, yolo11n + ByteTrack conf 0.3, 문 선 여유 폭 0.25m, 프레임 간격 0.2초. 보정 조건 = 직접 계수가 3초 동안 같은 값 + 문 근처에 아무도 없음. seed 1. 2026-11-03, `vhybrid.py`*.*

- 출입 계수: IN 26/30, OUT 17/26. 놓친 OUT이 더 많아 값이 계속 위로 밀렸다(편향 +1.69).
- 혼합: 보정이 값을 바꾼 것이 10번이었다(예: 120초 5→4, 150초 7→8). 하루 끝 드리프트는 0.
- 약점: 사람이 13~14명으로 많을 때 직접 계수가 가려진 사람을 덜 센다(편향 −1.07). 그때 보정하면 혼합도 따라 틀린다(369초 13→10, 정답 12). 개선안: 직접 계수가 믿을 만한 때(인원이 적을 때, 박스끼리 겹침이 적을 때)만 보정하거나, 보정 폭을 한 번에 ±1명으로 제한한다.

과제 5 심화 — 기준선 6cm vs 12cm (가상 시험)

기준선을 2배로 늘리자 거리 오차가 평균 약 2.2배 줄어 $\Delta Z \propto 1/B$가 대체로 성립했고, 거리가 멀수록 오차가 커졌다. 12cm는 모든 거리에서 기준을 통과했지만 6cm는 2m에서 기준(5%)을 넘었다.

거리 Z (m)	6cm 평균 오차 · 오차율	12cm 평균 오차 · 오차율	오차 비 (6cm ÷ 12cm)	이론 ΔZ 6cm / 12cm (실측 Δd)
1	0.041m · 4.1%	0.025m · 2.5%	1.7	0.055m (3.18px) / 0.024m (2.72px)
2	0.109m · 5.4%	0.045m · 2.3%	2.4	0.126m (1.82px) / 0.048m (1.40px)
3	0.240m · 8.0%	0.113m · 3.8%	2.1	0.303m (1.94px) / 0.129m (1.64px)
4	0.177m · 4.4%	0.182m · 4.5%	1.0	0.227m (0.82px) / 0.207m (1.49px)
5	0.422m · 8.4%	0.106m · 2.1%	4.0	0.407m (0.94px) / 0.132m (0.61px)

가상 시험(합성 영상) — 평행 배치 카메라 2대(1280x720, 초점거리 960px, 높이 1.2m, 기울기 0°), 사람 3명(키 1.7m), 화면마다 독립 센서 노이즈 σ=6, yolo11n-pose 어깨 중점의 시차로 Z = f·B/d, 거리마다 12번(좌우 위치 무작위), 모두 검출 성공(12/12). 이론 ΔZ = Z²·Δd/(f·B), Δd = 시차의 표준편차. 2026-11-03, `vbaseline.py`.

- 측정 오차가 이론식(실제로 잰 Δd를 넣은 값)과 거의 같다. 오차는 시차의 흔들림(키포인트가 1~3px 흔들림)에서 온다.
- 4m(비 1.0)와 5m(비 4.0)는 거리마다 12번뿐이라 흔들렸다. Δd가 거리마다 0.6~3.2px로 일정하지 않은 것도 원인이다.
- 선택: 사람 거리 측정에는 기준선 12cm 이상. 다만 기준선이 길수록 가까운 물체는 두 화면에 함께 보이지 않으므로 최소 거리가 늘어난다.
- 한계: 합성 영상이라 렌즈 흐림, 두 카메라 시간 차, 보정 오차가 없다. 실제로는 이보다 오차가 크다. 교육장에서 줄자로 1/2/3/4m를 표시해 실측 비교해야 한다.

확인 문제 3

3-1. 1920px 폭 영상, 맨 뒷자리 사람 키 70px

- 입력 640, 조각 없음: 70 × 640 ÷ 1920 ≈ 23px → 안정 기준 32px에 못 미친다.
- imgsz 960: 70 × 960 ÷ 1920 = 35px, 또는 입력 640 + 조각 2x2: 70 × 640 ÷ 960 ≈ 47px. 둘 중 하나면 32px을 넘는다(둘 다 쓰면 70px).
- 과제 3 시험에서도 입력 640 → 960이 MAE를 13.75 → 10.35명으로 줄였다.

3-2. 편향 −0.4

- 편향 = 센 값 − 정답이므로 음수는 덜 센다는 뜻이다.
- 먼저 할 조치: ① conf를 낮춘다(0.4 → 0.25, 과제 3에서 MAE 16.2 → 12.8), ② 입력 크기를 키우거나 조각 검출로 작은 사람을 살린다. 그래도 남는 꾸준한 누락은 직선 보정식으로 바로잡는다.

3-3. `--stat median` 과 `p75`

- median은 창 안 값의 가운데, p75는 아래에서 75% 지점 값이라 더 큰 쪽을 따른다.
- 책상·파티션에 가려지면 사람을 놓칠 뿐 없는 사람을 더 세지는 않는다. 그래서 창 안의 낮은 값은 가림 때문일 가능성이 커서, 높은 쪽(p75)이 실제에 가깝다.

3-4. 출입 계수에서 하루 끝 재실 −3

- 원인 ①: 나갈 때 선 위에서 흔들리거나 추적 ID가 바뀌어 OUT을 두 번 셈다(과제 2에서 여유 폭 없이 IN 7/OUT 6이 나온 것과 같다).
- 원인 ②: 여러 명이 겹쳐 들어오거나 다른 문으로 들어와 IN을 놓쳤다.
- 대책: 선 양쪽 여유 폭(히스테리시스), 모든 출입구에 카메라, 방이 비는 시간(새벽)에 0으로 초기화하고 음수는 0으로 자른다. 직접 계수로 주기적으로 누적 값을 바로잡는다.

3-5. MQTT `retain` 을 켜지 않으면

- 재시작한 에이전트는 다음 발행(우리 서비스는 최대 10초)까지 현재 인원을 모른다. 그동안 값이 없거나 0으로 오해해 제어를 잘못할 수 있다. retain을 켜면 브로커가 마지막 값을 접속 즉시 보내 준다.

3-6. 조각 검출(2x2)에서 경계에 걸친 사람

- 조각별 박스를 전체 화면 좌표로 옮긴 뒤 한 번 더 NMS(같은 사람으로 겹치는 박스 중 하나만 남기기)를 한다.
- 그래도 겹침 0.15를 두는 이유: 겹침이 없으면 경계의 사람이 반쪽씩 잘려 어느 조각에서도 제대로 검출되지 않는다. 겹치게 자르면 적어도 한 조각에는 온전하게 들어간다.

확인 문제 4

4-1. 특징 벡터 53차원

- 관절 17개의 (x, y) 좌표 34 + 관절별 신뢰도 17 + 정면 응시 점수 1 + 이동 속도 1 = 53.
- 신뢰도를 넣는 이유: 가려지거나 화면 밖인 관절의 좌표는 엉터리다. 신뢰도가 있어야 모델이 어느 좌표를 믿을지 배운다.

4-2. 손 흔들기 규칙의 오판·놓침

- 오판: 팔을 든 채 칠판을 지우거나 머리를 긁을 때도 손목이 좌우로 반복돼 손흔들기로 판정된다.
- 놓침: 옆모습이나 뒷모습이면 손목의 x좌표 변화가 작고, 어깨 아래에서 손만 작게 흔들거나 1.5초보다 느리게 흔들면 못 잡는다. 9단계에서는 연속으로 흔들면 1회로만 기록됐다.

4-3. 나란히 선 두 사람의 악수 오판 줄이기

- 두 사람의 몸통이 서로 마주 보는지(어깨선 방향·코 방향)를 조건에 더한다.
- 맞닿은 손목이 두 사람 골반 중심 사이에 있고, 0.5초 이상 유지되며 위아래로 작게 흔들리는지도 본다.

4-4. `--window 30 --stride 5` 와 검증 분할

- 학습 샘플 하나 = 연속 30프레임(30fps면 약 1초), 다음 샘플은 5프레임 뒤에서 시작한다.
- 이웃 샘플은 30프레임 중 25프레임이 같다. 무작위로 섞으면 거의 같은 샘플이 학습과 검증에 나눠 들어가 정확도가 부풀려진다. 그래서 시퀀스 뒷부분 20%를 따로 떼어 검증한다.

4-5. '관심 있음' 300건 vs 실제 멈춘 사람 50명

- 가장 의심되는 원인은 추적 ID가 끊기면서 같은 사람이 여러 번 새 사람으로 세인 것이다. 10단계에서도 위치를 못 잡자 2초마다 등장·퇴장이 반복됐다. 사람(ID) 단위로 중복을 없애고, 잠깐 사라진 ID는 일정 시간 안에 다시 잇는다.

확인 문제 5

5-1. f = 1000px, B = 0.08m, d = 40px

- $Z = f \cdot B \div d = 1000 \times 0.08 \div 40 = 2.0\text{m}$.

- $\Delta Z \approx Z^2 \cdot \Delta d \div (f \cdot B) = 4 \times 0.5 \div 80 = 0.025m$ = 약 2.5cm.

5-2. 거리 4m

- 시차 d = 80 ÷ 4 = 20px, ΔZ = 16 × 0.5 ÷ 80 = 0.1m = 약 10cm.
- 1m(d = 80px, ΔZ ≈ 0.6cm)와 비교하면 거리 4배에 오차는 16배(거리의 제곱)다.

5-3. `--num-disp 64`, scale 0.5일 때 최소 거리

- 최대 시차 = 64 ÷ 0.5 = 128px, 최소 거리 = 80 ÷ 128 ≈ 0.63m.
- 0.5m를 재려면 시차 160px이 필요하다 → num-disp를 80 이상(16의 배수인 96)으로 올리거나 기준선을 줄인다.

5-4. `--board 9x6`

- 체커보드 안쪽 꼭짓점(검은 칸 4개가 만나는 점)의 가로×세로 개수다. 10x7칸 보드의 안쪽 꼭짓점은 9x6이다.
- `--board 10x7`로 넣으면 그런 배치가 없어 체커보드를 한 장도 찾지 못한다. 집 실습에서도 10x7칸 이미지를 만들어 9x6으로 검출했다.

5-5. 재투영 0.3px인데 기준선이 80mm → 96mm

- 칸 크기(`--square`)를 잘못 넣었을 가능성이 가장 크다. 재투영 오차는 모양만 보므로 축척이 1.2배 틀려도 작게 나온다. 실제 칸을 자로 다시 재서 넣는다.

5-6. 무늬 없는 흰 벽이 검게(측정 불가) 나오는 이유

- SGBM은 왼쪽 영상의 작은 조각과 가장 비슷한 조각을 오른쪽 영상의 같은 행에서 찾는다. 흰 벽은 어느 조각이나 똑같아 비용이 평평하고, 유일성 검사를 통과하지 못해 측정 불가로 나온다.

확인 문제 6

6-1. GPU 점유율과 LLM

- 재실 1회 × 10ms + 통행 15회 × 5ms + 거리 15회 × 5ms = 1초에 160ms → 약 16%(SGBM 20ms는 CPU).
- LLM이 질의마다 GPU를 2초씩 쓰면 그동안 비전 추론이 밀려 프레임을 건너뛰거나 지연이 튀고, 통행 선 통과를 놓칠 수 있다.
- 완화: 최신 1장 리더로 밀린 프레임을 버리고, 작은·양자화된 LLM을 쓰며, 질의를 큐에 넣어 한 번에 하나만 처리하고 답 길이를 제한한다.

6-2. 재부팅 때 초기화되는 USB 웹캠 설정

- systemd의 `ExecStartPre=` 지시어로 서비스 시작 전에 설정을 다시 넣는다.
- 예: `ExecStartPre=/usr/bin/v4l2-ctl -d /dev/video0 -c focus_automatic_continuous=0 -c focus_absolute=0 -c power_line_frequency=2` (자동 초점 끄기, 초점 고정, 60Hz 깜빡임 방지).

6-3. 두 프로그램이 모두 `:8080`을 열어 두 번째가 죽음

- ① 프로그램마다 포트를 옵션으로 다르게 준다. 이번 실습에서도 8001·8086·8090이 이미 쓰여 8011·8096·8095로 바꿨다.
- ② 리버스 프록시(nginx) 하나가 8080을 열고 경로별로 나눠 전달한다. 시작 전에 포트가 비었는지 확인하는 것도 도움이 된다.

6-4. 영상이 보드 밖으로 나가는 경로와 막는 법

경로	운영 모드에서 막는 법
웹 뷰어(MJPEG)	끄거나 localhost에만 열기
RTSP 재송출	재송출 없음
스냅샷	저장하지 않음, `/health`의 saved_images = 0으로 확인
녹화	끄기
크롭(재라벨·임베딩용)	메모리에서만 쓰고 파일로 남기지 않기
로그·MQTT	숫자만(인원·상태), 이미지나 base64 없음

과제 6 서비스는 이 방식으로 인원 숫자만 내보낸다. 실습용 `vocab2.py`는 스냅샷을 저장하므로 운영에는 쓰지 않는다.

6-5. `count`와 `raw`

- 제어 로직은 창 평활된 `count`를 쓴다. 한 프레임의 가림·오검출로 값이 튀어도 냉난방·환기가 오락가락하지 않는다.
- `raw`는 지금 순간 값이다. 평활이 늦게 반응하는지 점검하고, 창 크기를 정하고, 카메라 이상을 찾는 데 쓰려고 남긴다.